

TP Queue : Projet de tâches avec files d'Attente

1. Préparation du projet Laravel

1. Créer un projet Laravel :

Ouvrez votre terminal et exécutez la commande suivante :

```
composer create-project laravel/laravel 026-queue-tasks
```

2. Accéder au projet :

Naviguez vers le répertoire du projet :

```
cd 026-queue-tasks
```

2. Configuration de la Base de Données (en cas de MYSQL)

1. Créer une base de données :

Créez une base de données MySQL pour votre projet.

2. Configurer .env :

Ouvrez le fichier .env et configurez les informations de connexion à la base de données :

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=laravel_queue
DB_USERNAME=root
DB_PASSWORD=
```

3. Création d'une Migration et d'un Modèle

1. Créer une migration :

Exécutez la commande suivante pour créer une table tasks :

```
php artisan make:migration create_tasks_table
```

2. Modifier la migration :

Ouvrez le fichier de migration dans **database/migrations** et modifiez-le comme suit :

```
public function up()
{
    Schema::create('tasks', function (Blueprint $table) {
        $table->id();
        $table->string('name');
        $table->text('description');
        $table->boolean('is_completed')->default(false);
        $table->timestamps();
    });
}
```

3. Exécuter la migration :

Exécutez la migration pour créer la table :

```
php artisan migrate
```

4. Créer un modèle :

Créez un modèle Task :

```
php artisan make:model Task
```

5. Ajouter \$fillable au modèle :

Ouvrez le fichier **Task.php** dans **app/Models** et ajoutez :

```
protected $fillable = [  
    'name',  
    'description',  
    'is_completed'  
];
```

4. Création d'un Job

1. Créer un Job :

Créez un Job pour exécuter la tâche en arrière-plan :

```
php artisan make:job ProcessTask
```

2. Modifier le Job :

Ouvrez le fichier **ProcessTask.php** dans **app/Jobs** et modifiez la méthode **handle** :

```
class ProcessTask implements ShouldQueue  
{  
    use Queueable;  
  
    protected $task;  
  
    public function __construct(Task $task)  
    {  
        $this->task = $task;  
    }  
  
    public function handle()  
    {  
        // Simuler une tâche qui prend du temps  
        sleep(10);  
  
        // Marquer la tâche comme terminée  
        $this->task->update(['is_completed' => true]);  
    }  
}
```

5. Création d'un Contrôleur et d'une Route

1. Créer un contrôleur :

Créez un contrôleur pour gérer les tâches :

```
php artisan make:controller TaskController
```

2. Modifier le contrôleur :

Ouvrez le fichier **TaskController.php** dans **app/Http/Controllers** et ajoutez :

```
public function index()
{
    $tasks = Task::all();
    return view('tasks.index', compact('tasks'));
}

public function store(Request $request)
{
    $task = Task::create([
        'name' => $request->input('name'),
        'description' => $request->input('description'),
    ]);

    // Dispatch the job to the queue
    ProcessTask::dispatch($task);

    return redirect()->route('tasks.index')->with('success', 'Task submitted successfully!');
}
```

3. Créer des routes :

Ouvrez le fichier **routes/web.php** et ajoutez :

```
Route::get('/tasks', [TaskController::class, 'index']->name('tasks.index'));
Route::post('/tasks', [TaskController::class, 'store']->name('tasks.store'));
```

6. Création des Vues

1. Créer une vue :

Créez un fichier **index.blade.php** dans **resources/views/tasks** et ajoutez :

```
<!DOCTYPE html>
<html lang="en">

<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Tasks</title>
    <link href="https://cdn.jsdelivr.net/npm/tailwindcss@2.2.19/dist/tailwind.min.css"
rel="stylesheet">
</head>

<body class="bg-gray-100">
    <div class="container mx-auto p-4">
        <h1 class="text-3xl font-bold text-center mb-6">Tasks</h1>

        <!-- Success Message -->
        @if (session('success'))
            <div class="bg-green-100 border border-green-400 text-green-700 px-4 py-3
rounded mb-4">
                {{ session('success') }}
            </div>
        @endif
```

```

<!-- Task Submission Form -->
<div class="bg-white p-6 rounded-lg shadow-md mb-8">
    <form action="{{ route('tasks.store') }}" method="POST">
        @csrf
        <div class="mb-4">
            <label for="name" class="block text-gray-700 font-bold mb-2">Task
Name:</label>
            <input type="text" name="name" id="name"
                class="w-full px-4 py-2 border rounded-lg focus:outline-none
focus:ring-2 focus:ring-blue-500"
                required>
            </div>
            <div class="mb-4">
                <label for="description" class="block text-gray-700 font-bold mb-
2">Task Description:</label>
                <textarea name="description" id="description"
                    class="w-full px-4 py-2 border rounded-lg focus:outline-none
focus:ring-2 focus:ring-blue-500" rows="4"
                    required></textarea>
            </div>
            <button type="submit"
                class="bg-blue-500 text-white px-4 py-2 rounded-lg hover:bg-blue-600
focus:outline-none focus:ring-2 focus:ring-blue-500">
                Submit Task
            </button>
        </form>
    </div>

<!-- Task List -->
<div class="bg-white p-6 rounded-lg shadow-md">
    <h2 class="text-2xl font-bold mb-4">Task List</h2>
    <ul>
        @foreach ($tasks as $task)
            <li class="mb-4 p-4 border rounded-lg {{ $task->is_completed ? 'bg-
green-50' : 'bg-gray-50' }}">
                <div class="flex justify-between items-center">
                    <div>
                        <h3 class="text-lg font-semibold">{{ $task->name }}</h3>
                        <p class="text-gray-600">{{ $task->description }}</p>
                    </div>
                    <span
                        class="px-3 py-1 text-sm font-semibold rounded-full {{
$task->is_completed ? 'bg-green-200 text-green-800' : 'bg-gray-200 text-gray-800' }}">
                        {{ $task->is_completed ? 'Completed' : 'Pending' }}
                    </span>
                </div>
            </li>
        @endforeach
    </ul>
</div>
</div>
</body>
</html>

```

7. Configuration de la File d'Attente

1. Configurer la file d'attente : (dans le cas de Laravel 9)

Exécutez la commande suivante pour créer la table des jobs :

```
php artisan queue:table  
php artisan migrate
```

2. **Configurer .env :**

Assurez-vous que votre fichier .env est configuré pour utiliser la file d'attente de base de données :

```
QUEUE_CONNECTION=database
```

8. Exécution de la File d'Attente

1. **Démarrer le worker :**

Exécutez la commande suivante pour traiter les jobs en attente :

```
php artisan queue:work
```

9. Tester le Projet

1. **Soumettre une tâche :**

Accédez à <http://localhost:8000/tasks>, soumettez une tâche et observez son traitement en arrière-plan.

2. **Vérifier la tâche :**

Rafraîchissez la page pour voir la tâche marquée comme terminée après 10 secondes.